

GCP Automated Access Review

Study Guide: AWS vs GCP comparison, lessons learned, and interview Q&A

Companion document to the working GCP build. Use this to internalize how GCP services map to their AWS counterparts, what design decisions changed and why, what gotchas we hit during the actual build, and how to talk about all of it in an interview.

Sections: **1.** Service-by-service comparison. **2.** Architecture similarities and differences. **3.** Concept-level differences (IAM, OIDC vs WIF, state). **4.** Lessons learned during this build. **5.** Interview Q&A nuggets specific to GCP and multi-cloud.

1. Service-by-Service Comparison

Direct mapping table. Use this to translate AWS architecture vocabulary into GCP architecture vocabulary on the fly.

Layer	AWS service	GCP equivalent	Key difference
Compute	Lambda	Cloud Functions 2nd gen	GCP CF 2nd gen runs on Cloud Run under the hood. Triggers via
Object storage	S3	Cloud Storage (GCS)	GCS encrypts at rest by default. AWS needs explicit SSE config.
Identity	IAM (inline policies)	Cloud IAM (role bindings)	GCP binds principals to predefined roles. No inline JSON policies
Scheduler	EventBridge	Cloud Scheduler + Pub/Sub	GCP needs an intermediate Pub/Sub topic; AWS EventBridge inv
AI / GenAI	Bedrock (Claude)	Vertex AI (Gemini)	Different model IDs, different SDK. Both pay-per-token.
Email	SES	No native equivalent	GCP has no managed transactional email service. We use Gmail
Audit log	CloudTrail	Cloud Audit Logs	Both always-on. Cloud Audit Logs split into Admin Activity vs Data
Findings hub	Security Hub	Security Command Center	SCC Premium tier needed for full features. Project-level access o
External access	IAM Access Analyzer	Recommender API + SCC	Split across two services in GCP.
Org hierarchy	Organizations + SCPs	Organization Policies	Both tree-structured. SCPs map to Org Policy Constraints.
State backend	S3 + use_lockfile	GCS backend	GCS has native locking via object generations, no flag needed.
CI auth	OIDC + STS AssumeRole	Workload Identity Federation	Same conceptual model. Different terminology, more verbose setu
Secret store	Secrets Manager	Secret Manager	Same name, near-identical model. GCP uses 'versions' explicitly.
Pipeline gate	OPA / Conftest	OPA / Conftest	Identical tool. Only the Rego rules change to target different resou

2. How the Two Systems Work

Both projects deliver the same outcome: a scheduled access review that pulls security findings from native cloud services, summarizes them with an LLM, archives a CSV report, and emails it to a recipient. The skeleton is identical. The implementations differ at every layer because the underlying services and integration patterns are different.

Similarities (the parts that are basically the same)

- **The skeleton:** scheduler triggers compute on a cron, compute pulls findings, calls an LLM, writes to object storage, sends email.
- **Infrastructure as Code:** both use Terraform. The HCL syntax is identical; only the resource types change.
- **Policy as code:** both use OPA + Conftest with Rego. The policies look almost identical except for the targeted resource type names.
- **State management:** both use a versioned, encrypted object-storage backend (S3 in AWS, GCS in GCP) with locking enforced by the cloud.
- **CI/CD shape:** both use GitHub Actions with the same six stages: fmt, lint, plan, OPA gate, artifact upload, conditional apply on push to main.
- **OIDC for CI auth:** both use OpenID Connect federation. GitHub presents an OIDC token, the cloud exchanges it for short-lived credentials. No long-lived secrets stored in CI.

Differences (the parts that change between AWS and GCP)

- **Compute trigger pattern:** AWS EventBridge invokes Lambda directly. GCP requires Cloud Scheduler -> Pub/Sub topic -> Cloud Function via Eventarc. Three resources instead of two.
- **IAM model:** AWS uses inline policy JSON attached to roles. GCP uses bindings (principal + role + scope). The Rego policy for least privilege has to target a different resource type to detect over-permissive access.
- **Email:** AWS has SES. GCP has no managed alternative. We use Gmail API with an OAuth refresh token stored in Secret Manager. Significantly more setup; a one-time manual OAuth flow is required.
- **Function source upload:** AWS Lambda accepts the zip directly with the function. GCP Cloud Functions 2nd gen requires the zip to live in a GCS staging bucket; the function references the GCS object.
- **Eventarc:** Cloud Functions 2nd gen with non-HTTP triggers actually goes through Eventarc, a separate service that needs its own API enabled. AWS Lambda has no equivalent indirection.
- **State locking:** AWS needs a separate DynamoDB table or use_lockfile=true (Terraform 1.10+). GCS backend has native locking with no flag needed and no separate resource.
- **Project-vs-account scope:** AWS uses accounts as the resource container. GCP uses projects. Multi-account in AWS is common; in GCP you typically have multiple projects under one organization.

3. Concept-Level Differences You Should Know Cold

AWS IAM vs GCP IAM

AWS pattern: a role has a trust policy (who can assume it) and one or more permissions policies attached. Permissions policies are JSON documents with Statement arrays containing Effect/Action/Resource. Inline policies are common.

GCP pattern: you bind a principal (user, group, service account) to a predefined or custom role at a resource scope (project, folder, org, individual resource). The 'policy' on a resource is the union of all bindings. There is no inline policy concept.

Practical implication: the AWS Rego policy targets `aws_iam_role_policy` resources and parses the inline JSON. The GCP equivalent targets `google_project_iam_member` resources and checks the role string directly. Same control intent, different resource types and shape.

AWS OIDC vs GCP Workload Identity Federation

Both let GitHub Actions assume cloud credentials without storing long-lived keys. The handshake is the same: GitHub mints an OIDC token; the cloud validates it and exchanges it for short-lived credentials.

AWS setup: OIDC identity provider + IAM role with trust policy. The trust policy contains a StringLike condition on the sub claim restricting which repo can assume the role. Three resources (provider, role, role policy).

GCP setup: Workload Identity Pool + Workload Identity Provider + service account + IAM binding allowing the WIF principal to impersonate the service account. Four resources, plus a separate `attribute_condition` on the provider AND a `principalSet` binding on the service account. Defense in depth: the provider rejects tokens from unauthorized owners, and the SA binding rejects impersonation from unauthorized repos.

Both are replay-resistant authentication. Both map to NIST 800-53 IA-2(8). The GCP version is more verbose but provides an extra layer of protection (the `attribute_condition` on the provider acts as a perimeter defense before the SA binding even gets evaluated).

State backend and locking

AWS: S3 backend stores the state file. Locking traditionally required a separate DynamoDB table (paying to keep a lock table just for Terraform). Terraform 1.10+ added `use_lockfile = true` which uses S3 conditional writes for native locking, removing the DynamoDB requirement.

GCP: GCS backend has had native locking since the beginning, via Cloud Storage object generation numbers. No separate lock table, no flag, no configuration. Just works.

Net: GCS state backend has fewer moving parts. AWS caught up with `use_lockfile`, but legacy projects still use DynamoDB for backwards compatibility.

Compute trigger patterns

AWS: EventBridge schedule directly targets Lambda. Two resources (rule + target) plus a Lambda permission for `events.amazonaws.com` to invoke. Linear: `schedule -> function`.

GCP: Cloud Scheduler publishes to Pub/Sub. Pub/Sub triggers Cloud Function 2nd gen via Eventarc. Five resources (scheduler job + topic + function event_trigger + Eventarc trigger created automatically + IAM binding for the Pub/Sub service to invoke). More plumbing, but the Pub/Sub topic is a useful fan-in point if you ever want multiple sources to trigger the function.

This is GCP being more decoupled at the cost of more resources to manage. For a single-source, single-target schedule, AWS's pattern is cleaner. For a multi-source orchestration with retries and dead-letter queues, GCP's pattern is more flexible.

4. Lessons Learned (carried from AWS, plus new ones from this GCP build)

These are the gotchas we actually hit. Memorize them; they double as 'tell me about a CI bug you debugged' interview answers.

Carried from the AWS build

- **CI hangs forever when a required variable has no default.** Locally satisfied by gitignored .tfvars; in CI there is no value, plan defaults to interactive prompt, runner has no TTY, plan hangs. Fix: -input=false on every plan call, plus pass values via TF_VAR_ env vars from secrets.
- **Workflow does not auto-trigger on the commit that introduces it** when path filters are present. Fix: include workflow_dispatch trigger so manual runs are always available.
- **State lock contention from concurrent or cancelled runs.** A cancelled run can leave a stale lock. Cancel cleanly; clear the lock object if needed before retrying.
- **Markdown auto-linking corrupts pasted code.** Anything that looks like a domain (data.aws, function.zip) gets turned into a markdown link. Fix: use the file-writing tool directly instead of copy-paste from chat.
- **IAM role/SA propagation takes time.** First Lambda or Cloud Function deploy can sit for 1-3 minutes while permissions propagate. Don't ctrl-C within the first 5 minutes.

New lessons from this GCP build

- **Eventarc API needed for Cloud Functions 2nd gen Pub/Sub triggers.** First terraform apply failed with 'Eventarc API has not been used.' Cloud Functions 2nd gen with non-HTTP triggers goes through Eventarc, a separate service with its own API to enable. Added eventarc.googleapis.com to bootstrap_gcp.sh.
- **Secret Manager versions must exist before a Cloud Function that mounts them can deploy.** The function had secret_environment_variables referencing version="latest". Cloud Run validates the secret reference at deploy time, but the secret had no versions yet (we planned to populate via gmail_oauth_setup.sh after deploy). Fix: seed a placeholder version before the function is created. The OAuth setup script overwrites it with the real refresh token later.
- **Vertex AI Gemini 2.0 needs explicit access.** First test invocation logged 'Publisher Model gemini-2.0-flash-001 was not found or your project does not have access.' Newer Gemini 2.0 models need approval via Vertex Model Garden. Fix: use gemini-1.5-flash-002 which is GA and accessible by default. The fallback summary kicked in so the email still sent; just no AI narrative until the model swap.
- **Python package import paths can be confusing.** The google-cloud-iam package does not expose google.iam.admin_v1 the way I assumed. Service account key listing is not in the stable public Python client. Fix: skip the SA key age check or implement via REST API. Don't trust autocomplete; verify against the package's actual __init__.py.
- **Security Command Center is org-level by default.** Project-level SCC requires the Premium tier. Free-tier projects fail SCC findings calls. The function caught the 404 non-fatally and continued; report still shipped with IAM and audit log findings.
- **OAuth 'Access blocked: Desktop did not complete verification' means you are not a test user.** External OAuth apps in Testing status only let listed test users authorize. Fix: add the email to the OAuth consent screen's test user list.

- **OAuth localhost redirect refusing to connect is expected behavior.** The redirect URI `http://localhost:8080` has nothing listening; the browser shows an error. The auth code is in the address bar and is what matters. Copy the URL from the address bar, not the page contents.

5. Interview Q&A;: GCP and Multi-Cloud Specific

Q1. Why did you build the same tool on both AWS and GCP?

Three reasons. First, multi-cloud GRC engineering is increasingly the norm; clients have workloads in both. Second, the comparison forces you to learn the underlying patterns rather than memorize cloud-specific syntax. Third, the policies and pipeline structure carry over with minimal change, so the GRC engineering value compounds across clouds.

Flex line: "The skeleton is identical; the implementation details translate. That's exactly the point of doing it twice."

Q2. What's the biggest architectural difference between the AWS and GCP versions?

The compute trigger pattern. AWS EventBridge invokes Lambda directly with a single rule + target + permission. GCP requires Cloud Scheduler -> Pub/Sub -> Cloud Function via Eventarc, which is more resources but introduces a useful fan-in point. The Pub/Sub topic could later accept events from multiple sources without changing the function.

Flex line: "GCP's pattern is more decoupled at the cost of more resources. AWS's pattern is simpler for the single-source case."

Q3. How does GCP IAM differ from AWS IAM in practice?

AWS uses inline JSON policies attached to roles. GCP uses role bindings: you bind a principal to a predefined or custom role at a resource scope. There's no inline policy concept. Practical impact: my Rego policy for least privilege targets a different resource type. AWS targets `aws_iam_role_policy` and parses the JSON; GCP targets `google_project_iam_member` and checks the role name.

Flex line: "Same control, same intent, different resource types. The Rego is almost identical line-for-line."

Q4. Walk me through Workload Identity Federation.

GitHub Actions has an OIDC issuer. The workflow requests a JWT from that issuer scoped to this run. The `google-github-actions/auth` action presents the JWT to GCP STS, which validates the signature against the registered Workload Identity Provider, checks the `attribute_condition` on the provider, and if it passes, allows impersonation of a service account. The service account's permissions become available to the workflow for one hour.

Flex line: "Two layers of restriction: the provider's `attribute_condition` rejects tokens from unauthorized repository owners, and the SA's IAM binding restricts impersonation to a specific repo. Defense in depth."

Q5. Why use Gmail API instead of a transactional email service?

GCP has no native managed email service like AWS SES. Options were Gmail API, SendGrid, or Mailgun. Gmail API uses my existing Gmail account, no third-party signup, and is free within my send limits. The trade-off is more setup: OAuth 2.0 flow with refresh token storage in Secret Manager. Acceptable for a demo and a small client; for production scale, switch to a transactional email service.

Flex line: "I picked the option with the most interesting auth setup specifically because it teaches the OAuth and Secret Manager patterns. SendGrid would have shipped faster but with less depth."

Q6. Why Gemini Flash over Claude on Vertex?

Three reasons. First, Gemini Flash is GCP-native, no model approval queue. Claude on Vertex requires Anthropic approval which can delay deployment. Second, Flash is right-sized for summarization; the larger Claude models would be 10x cost for marginal quality lift on a paragraph of findings. Third, removing the

Anthropic dependency simplifies the build.

Flex line: "Same reasoning as the AWS version where I picked Haiku over Opus. Match the model to the workload."

Q7. Tell me about a bug you debugged on the GCP build.

Cloud Function deployment failed with 'Secret was not found.' The function had `secret_environment_variables` referencing `version=latest`, but the Secret Manager resource had no versions yet. I planned to populate it via the OAuth setup script after deploy. Cloud Run validates the secret reference at deploy time, not runtime. Fix: seed a placeholder version before the function is created. The OAuth setup overwrites it with the real value later.

Flex line: "Lesson: `secret_environment_variables` validates at deploy time. Either create the version before the function, or read secrets at runtime via the SDK."

Q8. What did you skip and why?

Three things. The service account key age check was supposed to use `google-cloud-iam`, but the import path I assumed didn't exist; rather than implement via REST, I skipped the check. Security Command Center findings work at org level by default; I left it as a non-fatal skip for project-only deployments. Multi-environment promotion (dev/staging/prod) wasn't built; one project, one deployment for this scope.

Flex line: "Right-sizing is a maturity signal. I documented the skips in `lessons-learned.md` so future iterations know what's missing and why."

Q9. How would you scale this to multiple GCP projects or an org?

Two paths. Per-project: same Terraform deployed to multiple projects via workspace or per-project tfvars. Org-wide: one central project hosts the function and report bucket; the function's service account gets org-level `securityReviewer` and `assetViewer` roles, then iterates over projects in the org. The org-wide approach scales better but requires org admin permissions.

Flex line: "The Rego policies don't change. Same controls enforced regardless of how many projects you deploy to."

Q10. If I asked you to add a new compliance control tomorrow, walk me through what you'd do.

Open policy/, add a new `.rego` file in the same package style, with a deny rule that walks `input.resource_changes` and matches the violation pattern. Add the NIST 800-53 control number and the CMMC equivalent in a header comment. Run `confest test --policy ../policy --all-namespaces tfplan.json` locally to verify. Commit, PR, watch CI run. Total time: about 15 minutes for a simple rule.

Flex line: "The control IS the policy file. Adding controls is a code change, reviewable and testable."

How to use this document

Read it once end to end. Use the comparison table when an interviewer mentions one cloud and you need to answer in terms of the other. Memorize the lessons learned section and the bug debugging story; those are the most likely 'tell me about a time' questions.

Pair this with the AWS Q&A nuggets PDF. Together they cover both single-cloud and multi-cloud GRC engineering questions.

Good luck.